

COLEXIO VIVAS S.L.	RAMA:	Informática	CICLO:	DAM				
	MÓDULO	Programación de servicios y procesos					CURSO:	2º
	PROTOCOLO:	Apuntes clases	AVAL:		DATA:			
	AUTOR	Francisco Bellas Aláez (Curro)						

Ejercicios

Ejercicio 1

a) Crea dos hilos (Thread) compitiendo que traten de incrementar (el primer hilo) o decrementar (el segundo hilo) en 1 unidad una variable que comienza en 0. Funcionarán de forma continua hasta que a llegue a 500 (primer hilo) o a -500 (segundo hilo). Además se mostrará en pantalla la variable cada vez que cambie indicando quién la ha cambiado (thread 1 o thread 2 y cada uno en un color). Ambos hilos deben parar en cuanto uno consiga su objetivo. No uses `setCursorPosition`.

El Main, una vez que lanza los hilos, se queda en espera hasta que ambos hilos finalizan, luego informa de cual ha ganado.

b) Las funciones de hilos serán expresiones lambda (si quieres y los ves claro haz ya directamente este apartado).

Ejercicio 2

Realiza en consola el juego de carreras de caballos con al menos 5 caballos (haz un array de hilos) pero teniendo en cuenta que ahora cada caballo es un objeto de tipo Thread. El usuario hace su apuesta y luego empieza la carrera de caballos de forma que cada uno se mueve una distancia aleatoria y "duerme" un tiempo aleatorio. Al empezar la carrera el Main (o la función inicial) se queda en espera. Una vez que un caballo llega a la meta todos paran y el main continúa indicando el caballo ganador y si el usuario ha ganado. Se permitirá la repetición del juego. No uses expresiones lambda en este ejercicio.

Visualmente deben verse en varias líneas los caballos avanzar, cada uno en la suya. Puede ser tan simple como un asterisco que avance desde el principio de la línea hasta la meta.

Nota: De cara a realizar pruebas de este juego, se recomienda quitar la aleatoriedad temporalmente para forzar a que varios caballos lleguen a un tiempo y ver que solo uno es el que "cruza" la meta.

COLEXIO VIVAS S.L.	RAMA:	Informática	CICLO:	DAM				
	MÓDULO	Programación de servicios y procesos					CURSO:	2º
	PROTOCOLO:	Apuntes clases	AVAL:		DATA:			
	AUTOR	Francisco Bellas Aláez (Curro)						

Ejercicio 3

Realiza un simulador de descargas asíncronas en modo gráfico. Debes realizar una función Task que devuelva un string con el contenido supuestamente descargado. La cabecera de la función tendrá esta forma:

```
async Task<string> DownloadFileAsync(string fileName, int delayMs)
```

En realidad se simulan los tiempos usando Task.Delay para el retardo. Lo que realmente va a devolver la función es el mensaje "File *fileName* downloaded in *delay* ms.". Por supuesto *fileName* y *delay* serán los valores de los parámetros.

La aplicación WindowsForms dispondrá de un TextBox denominado txtFileName y un botón. Cada vez que se meta ahí una cadena y se pulse el botón o se pulse la tecla Enter se lanza una tarea con DownloadFileAsync usando el nombre de txtFileName y un retardo aleatorio entre 1 y 10 segundos. Cuando finaliza se añade la cadena devuelta a una nueva línea de otro TextBox multilinea (txtResults).

Por supuesto el programa no debe bloquearse nunca.

Ejercicio 4:

Realiza una aplicación a la cual se le pueda introducir una ruta o URL en un textbox (**txtUrl**) y disponga de los siguientes botones con las posibilidades indicadas:

Búsqueda: Este botón tiene asociado un TextBox donde se puede meter una cadena de caracteres. Al pulsarlo el programa buscará todas las apariciones de la cadena en todos los archivos *.txt que haya en el directorio indicado por txtUrl.

Para ellos sigue los siguientes pasos:

1. Haz una función denominada BuscaPalabra a la que le pasas una ruta y una cadena y busque la cantidad de veces que aparece la cadena en el archivo. Devuelve una cadena donde se indica el nombre del archivo, la palabra buscada y la cantidad de veces que aparece.

No hagas un algoritmo óptimo de búsqueda con split o expresiones regulares ya que se trata de ver que aunque la búsqueda sea lenta el programa sigue en funcionamiento. Haz una comparación exhaustiva del string barriendo cada substring del archivo. Incluso si lo consideras mete retardos.

COLEXIO VIVAS S.L.	RAMA:	Informática	CICLO:	DAM				
	MÓDULO	Programación de servicios y procesos					CURSO:	2º
	PROTOCOLO:	Apuntes clases	AVAL:		DATA:			
	AUTOR	Francisco Bellas Aláez (Curro)						

2. En el click del botón haz un bucle que recorra todos los archivos de texto del directorio y con cada uno lanza una tarea (Task.Run) que ejecute la función BuscaPalabra con cada archivo y añádela una colección de tareas (List<Task<string>>).
3. Haz un bucle que mientras haya tareas en la colección, espere a que acabe una (WhenAny), cuando esto suceda la eliminas de la colección y cojes el resultado de esa tarea y lo añades al ListBox. Como las otras tareas siguen en ejecución en la siguiente vuelta del bucle se espera a la siguiente.

Posición: Este también tiene asociado un TextBox (si quieres puede reutilizar el de búsqueda). Este indica la posición donde aparece la palabra indicada en el TextBox en cada uno de los archivos del directorio. Si no existe pone -1 un el ListBox de resultados. La diferencia que este hace todas las búsquedas y luego pone todos los resultados de golpe (WhenAll).

Haz en este caso una función que busque la primera aparición de una palabra y devuelva una cadena indicando Archivo, palabra y posición o que no existe (mete un Delay aleatorio para ralentizarla). Nuevamente evita optimizar algoritmos o incluso si quieres puedes introducir retardos.

Http: Este también tiene asociado un TextBox (si quieres puede reutilizar el de búsqueda). Este utiliza la clase HttpClient y muestra en la listbox lo que devuelve la función GetAsync pasándole como parámetro la URL que haya en el textbox.

Las entradas en el ListBox deben indicar: Botón pulsado y resultado.

Se permite la captura de excepciones genéricas en este ejercicio.

Puedes descargar textos largos de webs como : <https://www.gutenberg.org>

COLEXIO VIVAS S.L.	RAMA:	Informática	CICLO:	DAM				
	MÓDULO	Programación de servicios y procesos					CURSO:	2º
	PROTOCOLO:	Apuntes clases	AVAL:		DATA:			
	AUTOR	Francisco Bellas Aláez (Curro)						

Ejercicio 5:

Se desea hacer una simulación de un sistema productor consumidor mediante hilos de clase Thread. Para ello existe dos hilos, un productor y un consumidor. El productor va generando números aleatorios entre 1000 y 10000, lo muestra en pantalla y los añade al final de una colección. El consumidor va leyendo números del principio de la colección (y los extrae de la misma) y comprueba si son o no primos. Indica en pantalla el número y si es primo. Cuando haya detectado 5 primos el consumidor lo indica por pantalla y hace que el productor finalice, el consumidor sigue haciendo su tarea.

No uses SetCursor position. Usa distintos colores para info de consumidor y productor. Pon más prioridad al consumidor pues al procesar más es más lento.

El main debe esperar a que ambos hilos finalicen.

Ejercicio 6

En consola realiza mediante Tasks la búsqueda de un valor en un array bidimensional. Lanza una tarea por cada fila de forma que en cuanto una tarea encuentre el número en su fila ya lo indique por pantalla. Usa un array grande. Comprueba que la búsqueda es correcta mostrando al final dicha posición del array.

Valores ejemplo:

Número a buscar: 42

Tamaño array: 10x10000000

Números aleatorios dentro del array: de 1 a Cantidad de elementos/2.

COLEXIO **VIVAS** S.L.

RAMA:	Informática	CICLO:	DAM				
MÓDULO	Programación de servicios y procesos					CURSO:	2º
PROTOCOLO:	Apuntes clases	AVAL:		DATA:			
AUTOR	Francisco Bellas Aláez (Curro)						

OPCIONAL

(Revisa el apéndice de Trabajo con la clas Proccessprocesos)

Se desea realizar una versión del **TaskManager**. El programa planteará un formulario con una textbox amplia (pon propiedad multiline a true) donde mostrará información, otra textbox sencilla y los siguientes botones:

- **View Processes:** Mostrará todos los procesos en ejecución con la siguiente información: PID ,Nombre y titulo de la ventana principal si lo tiene. Aparecerá en columnas bien alineados en el textbox multilinea.

Al dar formato, si un nombre o título de ventana ocupa más que lo permitido debe "acortarse" y aparecer con puntos suspensivos. Por ejemplo: `Microsoft.ServiceHub.Controller` que aparezca como `Microsofts.S...` si el nombre lo tienes limitado a 15 caracteres. Trata de hacerlo con `Array.ForEach` con expresión lambda cuando sea posible.

- **Process info:** En la segunda textbox se puede meter un PID del proceso y se mostrará toda la información posible (que se haya explicado) sobre el proceso incluyendo subprocesos (su id y hora de comienzo) y módulos (nombre del módulo y nombre del archivo).
- **Close process:** Se introduce el PID en el textbox simple y hace una petición de cierre al programa.
- **Kill process:** Se introduce el PID en el textbox simple y se fuerza el cierre del proceso.
- **Run App:** A partir de su nombre o path y nombre en el textbox simple se ejecuta la aplicación indicada si es posible.
- **Stars With...:** Muestra los procesos cuyo nombre comienzan por lo que se indica en el textbox sencillo en la textbox amplia. *Trata de usar funciones vistas de la clase Array y expresiones lambda.*

Notas: Para hacer **retornos de carro en el TextBox** usa la propiedad **Environment.NewLine** o `"\r\n"`.

Además para que quede todo bien formateado debes buscar alguna **fuentes monoespaciada** como Lucida Console, Courier o similar.

Recuerda tener la aplicación en **64 bits y con permisos de Administración**.

COLEXIO **VIVAS** S.L.

RAMA:	Informática	CICLO:	DAM				
MÓDULO	Programación de servicios y procesos					CURSO:	2º
PROTOCOLO:	Apuntes clases	AVAL:		DATA:			
AUTOR	Francisco Bellas Aláez (Curro)						

Aunque no se pide, si quieres utilizar el `Array.ForEach` en otras colecciones como las de hilos se pueden hacer conversiones de esta forma:

```
p.Threads.Cast<ProcessThread>().ToArray()
```

Si aún lo quieres hacer más complejo:

Setup: Establece la configuración de lo que se puede ver de los procesos en la opción de visualización, a saber: PID, nombre, prioridad, título del formulario principal, nº de hilos, tiempo total de uso del procesador en segundos, memoria física y memoria virtual usada por el proceso. Esto se hará en un formulario secundario con varios checkboxes para elegir qué se debe visualizar (o puede estar en el propio formulario principal).

Usa un componente `ListView` con varias columnas (una para cada elemento de información) en lugar del `TextBox` y usa un `Timer` para que en el caso de tener alguna opción de información seleccionada que la actualice cada medio segundo. Puedes si lo deseas sustituir algunos botones por `radiobuttons`, `checkboxes` u otros componentes